



**College of Technology and Built Environment
School of Information Technology and Engineering
Department of IT/SE Eng.**

**LLM-Based Support Bot for Ethiopian Revenue Authority to enhance
information desk service for tax payers and business owners**

Software Design Specification

Team Members

1. Abdurahman Mohammed ...ATE/8901/13
2. Amanuel Ayalew.....ATE/3871/13
3. Basliel selamuATE/6761/13
4. Bethel Wondowssen.....ATE/8712/13
5. Diborah Dereje.....ATE/1712/13

Advisor: Mr Daniel

Date: Dec 27, 2025

SupportBot AI Next-Gen Revenue Customer Service

Table of Contents

- List of Tables.....2
- List of figures.....3
- Definitions, Acronyms, Abbreviations.....4
- 1. Introduction.....1**
 - 1.1 Purpose.....1
 - 1.2 General Overview.....1
 - 1.3Development Methods & Contingencies.....1
- 2. System Architecture.....2**
 - 2.1 Subsystem decomposition.....2
- 3. Object Model.....5**
 - 3.1 Class Diagram.....5**
 - 3.2 Sequence Diagram.....7
 - 3.3 State chartDiagram.....10
- 4. Detailed Design.....11**
 - 4.2 Class: Admin (Inherits from User).....13
 - 4.3 Class ChatSession.....15
 - 4.4 Class KnowledgeEngine (RAG Controller).....17
 - 4.5 Class WebScraper.....20
 - 4.6 Algorithm Design.....22
 - 4.6.1 Retrieval Augmented Generation (RAG) Flow.....22
 - 4.6.2 Vector Similarity Search (Cosine Similarity).....22
 - 4.6.3 Text Chunking Algorithm (Sliding Window).....23
- 5. References.....24**

SupportBot AI Next-Gen Revenue Customer Service

List of Tables

1. Table 4.1: Attributes description for Document class
2. Table 4.2: Operation description for Document class
3. Table 4.3: Attributes description for Admin class
4. Table 4.4: Operation description for Admin class
5. Table 4.5: Attributes description for ChatSession class
6. Table 4.6: Operation description for ChatSession class
7. Table 4.7: Attributes description for KnowledgeEngine class
8. Table 4.8: Operation description for KnowledgeEngine class
9. Table 4.9: Attributes description for WebScraper class
10. Table 4.10: Operation description for WebScraper class

SupportBot AI Next-Gen Revenue Customer Service

List of figures

1. Architecture Diagram
2. Class Diagram Part 1: Domain Layer (Data Model)
3. Class Diagram Part 2: Application Layer (Controllers & Services)
4. Class Diagram Part 3: Infrastructure Layer (External Interfaces)
5. Figure 1: Standard RAG Conversation Flow (Success Scenario)
6. Figure 2: Session Validation & Security Flow
7. Figure 3: Error Handling & Fallback Strategy
8. State Chart Diagram: System Logic Tree (Decision Flow)

SupportBot AI Next-Gen Revenue Customer Service

Definitions, Acronyms, Abbreviations

Acronym/Term	Definition	Context/Source
RAG	Retrieval Augmented Generation	Defined as the system's core method for answering questions based on facts ¹¹¹¹ .
LLM	Large Language Model	Described as the AI component used to generate responses.
MoR	Ministry of Revenue	The source of the regulatory documents ingested by the system ³ .
OOD	Object-Oriented Design	The methodology used to model interactions between Users, Documents, and the Chat Engine.
RBAC	Role-Based Access Control	The architecture used to enforce boundary controls and privileges.
JWT	JSON Web Token	The token format used to validate and authorize Administrator sessions.
CRUD	Create, Read, Update, Delete	The set of capabilities granted to Administrators regarding the Knowledge Base.
OCR	Optical Character Recognition	Listed as a component of the DocumentProcessor within the Ingestion Subsystem.
UUID	Universally Unique Identifier	The data type used for unique identification of documents and sessions.

1. Introduction

1.1 Purpose

The purpose of this System Design document is to translate the business requirements and business processes defined in the SRS into a technical design that will be used to develop the LLM-Based Support Bot for the Ethiopian Revenue Authority. This document describes the system architecture, subsystem decomposition, and detailed object models required to implement the advanced Retrieval Augmented Generation (RAG) system with hybrid search capabilities.

1.2 General Overview

The system is designed as an AI-powered information desk agent accessible via web interface and Telegram bot, capable of answering tax-related queries in Amharic and English. Unlike generic RAG implementations, this system employs a **hybrid retrieval architecture** combining BM25 keyword matching with dense vector search, specifically optimized for Ethiopian tax terminology and Amharic language processing.

The system operates by:

1. Ingesting regulatory documents from the Ministry of Revenue (MoR) website
2. Converting documents into vector embeddings using multilingual models (multilingual-e5-large)
3. Implementing dual retrieval paths (BM25 + semantic) with Reciprocal Rank Fusion
4. Using a Large Language Model (LLM) to generate regulation-grounded responses with mandatory citations
5. Applying confidence scoring to prevent hallucinations and trigger appropriate fallbacks

1.3 Development Methods & Contingencies

Methodology: The system design utilizes **Object-Oriented Design (OOD)** principles to model the interaction between Users, Documents, and the Chat Engine. The development follows an **Agile** approach with iterative sprints, allowing for rapid adjustment of the LLM prompts and retrieval parameters.

Technologies:

1. **Backend:** Python (FastAPI) for high-performance API handling.
2. **Frontend:** Next.js (React with TypeScript) for responsive web interface and Telegram bot integration.

SupportBot AI Next-Gen Revenue Customer Service

3. **AI/ML:** LangChain for orchestration, Gemini 2.5 Flash for inference, multilingual-e5-large for embeddings.
4. **Database:** PostgreSQL with pgvector extension for vector storage and built-in full-text search for BM25-style keyword matching.
5. **NLU:** XLM-RoBERTa for intent classification, fastText for language detection.

Contingencies:

1. **LLM Latency:** If external APIs (e.g., OpenAI) experience high latency (>5s), the design supports swapping the LLMService interface for a local, quantized Llama 3 8B model running on the host server.
2. **Scraper Blocking:** If the MoR website changes its DOM structure, the Scraper module is isolated as a microservice. This allows the team to update the scraping logic without redeploying the core chat.
3. **Low Confidence Responses:** If retrieval confidence falls below 0.6 threshold, the system returns a standardized fallback message rather than potentially hallucinated content.

2. System Architecture

2.1 Subsystem decomposition

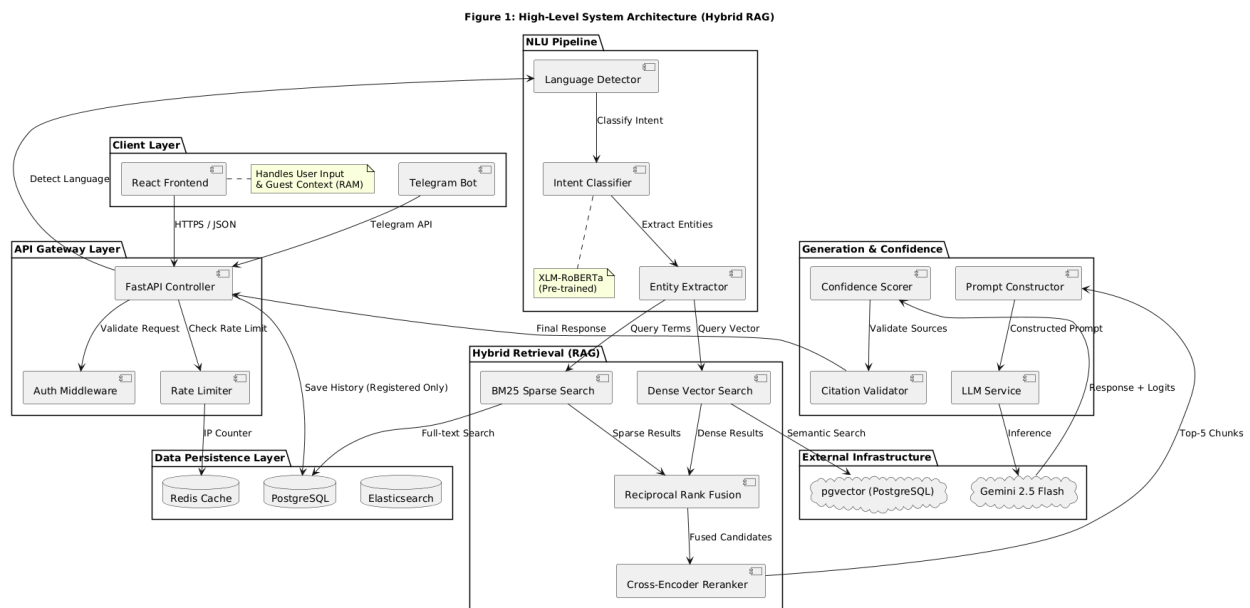
The system is decomposed into four major subsystems to ensure separation of concerns.

1. **Client Subsystem (Frontend):**
 1. Responsible for rendering the chat interface, handling user input, and displaying markdown responses.
 2. **Components:** ChatUI, HistoryManager, APIConnector.
2. **API Subsystem (Backend Controller):**
 1. Acts as the central controller, managing session state and routing requests.
 2. **Components:** NestJS, SessionManager, RateLimiter.
3. **Intelligence Subsystem (RAG Engine):**
 1. Responsible for semantic search and answer generation.
 2. **Components:** EmbeddingEngine, VectorStoreClient (Pinecone), LLMClient.
4. **Ingestion Subsystem (Data Pipeline):**
 1. Background workers that keep the knowledge base current.
 2. **Components:** WebScraper, DocumentProcessor (OCR), ChunkingService.

SupportBot AI Next-Gen Revenue Customer Service

Hybrid Storage Architecture The system employs a dual-storage strategy to handle different user types efficiently:

- **Persistent Storage (PostgreSQL):** Used exclusively for Registered Users to store profiles and long-term chat history.
- **Volatile Storage (Redis + RAM):** Used for Guest Users. Redis tracks IP addresses for rate limiting (with a 10-minute Time-To-Live), while the application RAM holds the active conversation context. This ensures guest data is naturally purged when the session ends.



SupportBot AI Next-Gen Revenue Customer Service

2.2 Hardware/Software Mapping

The system architecture abstracts the software layer from the underlying hardware through a containerized deployment strategy. While the application is inherently platform-independent, it is explicitly optimized for a Docker environment. This approach ensures operational consistency across development, testing, and production stages by encapsulating all necessary dependencies within the container. Consequently, the system can be deployed seamlessly on any host operating system that supports containerization, eliminating hardware-specific compatibility issues.

2.3 Access Control

To ensure data integrity and system security, the platform enforces strict boundary controls using a Role-Based Access Control (RBAC) architecture. This model clearly delineates privileges between transient public interactions and privileged administrative operations.

Public User Access The Public User role is designed to prioritize accessibility and ease of use for taxpayers. To lower the barrier to entry, the system utilizes anonymous authentication, assigning a unique Session ID managed via browser cookies rather than requiring formal login credentials. The permissions for this role are strictly scoped to "read-only" interactions within the Chat Interface. This allows users to submit queries and review their specific session history, while ensuring they cannot modify the system's underlying configuration or data.

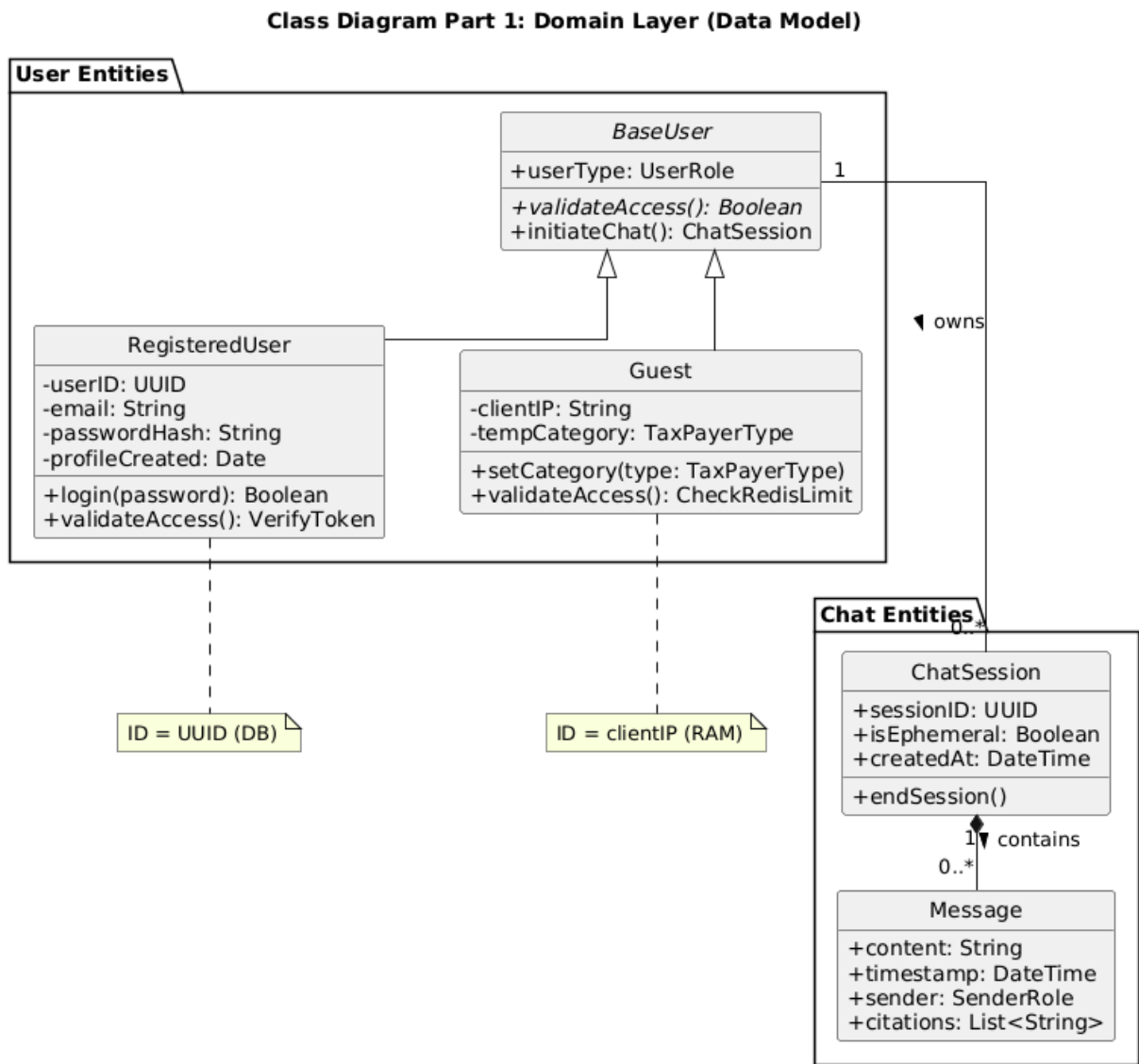
Administrator Access In contrast, the Administrator role is granted comprehensive control over the system's operational and data layers. Access to this role is secured through a robust authentication process requiring valid credentials (username and password), which are subsequently validated via a JSON Web Token (JWT). Administrators possess full CRUD (Create, Read, Update, Delete) capabilities regarding the Knowledge Base. Their operational scope extends to critical maintenance tasks, including the manual ingestion of documents, auditing of error logs, execution of manual scraping triggers, and the mitigation of security threats through IP blacklisting.

SupportBot AI Next-Gen Revenue Customer Service

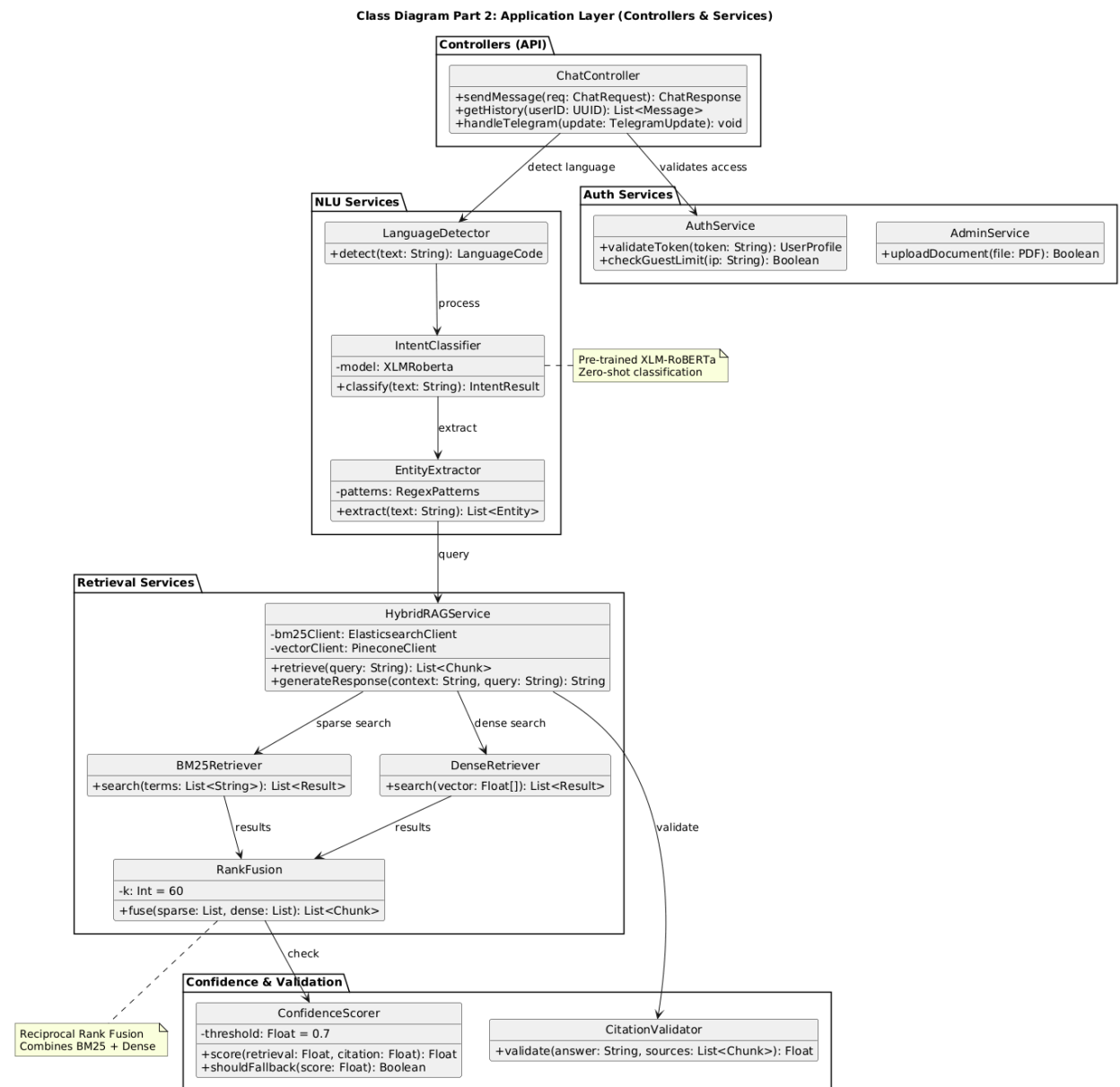
3. Object Model

3.1 Class Diagram

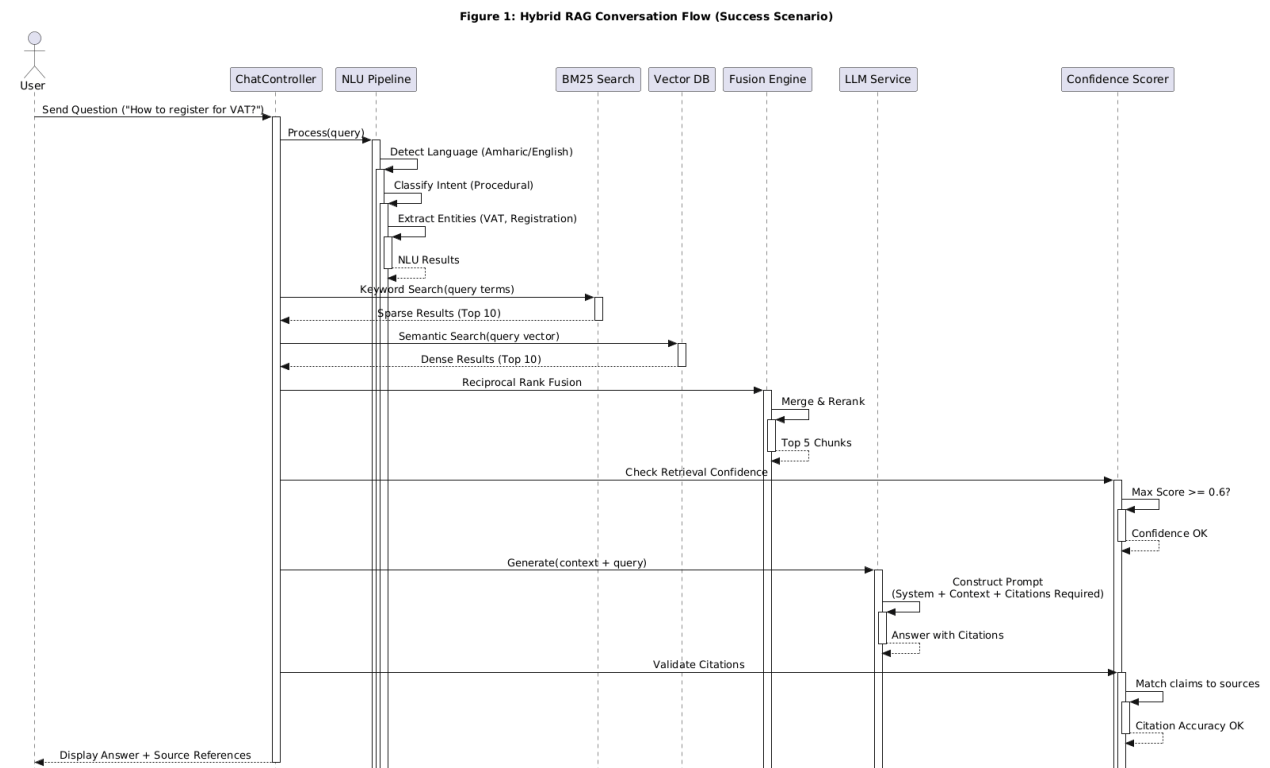
This section provides the detailed design for the classes identified in the Class Diagram (Section 3.1). It defines the specific attributes, types, visibility, and contractual obligations (pre/post-conditions) for the core system components.



SupportBot AI Next-Gen Revenue Customer Service

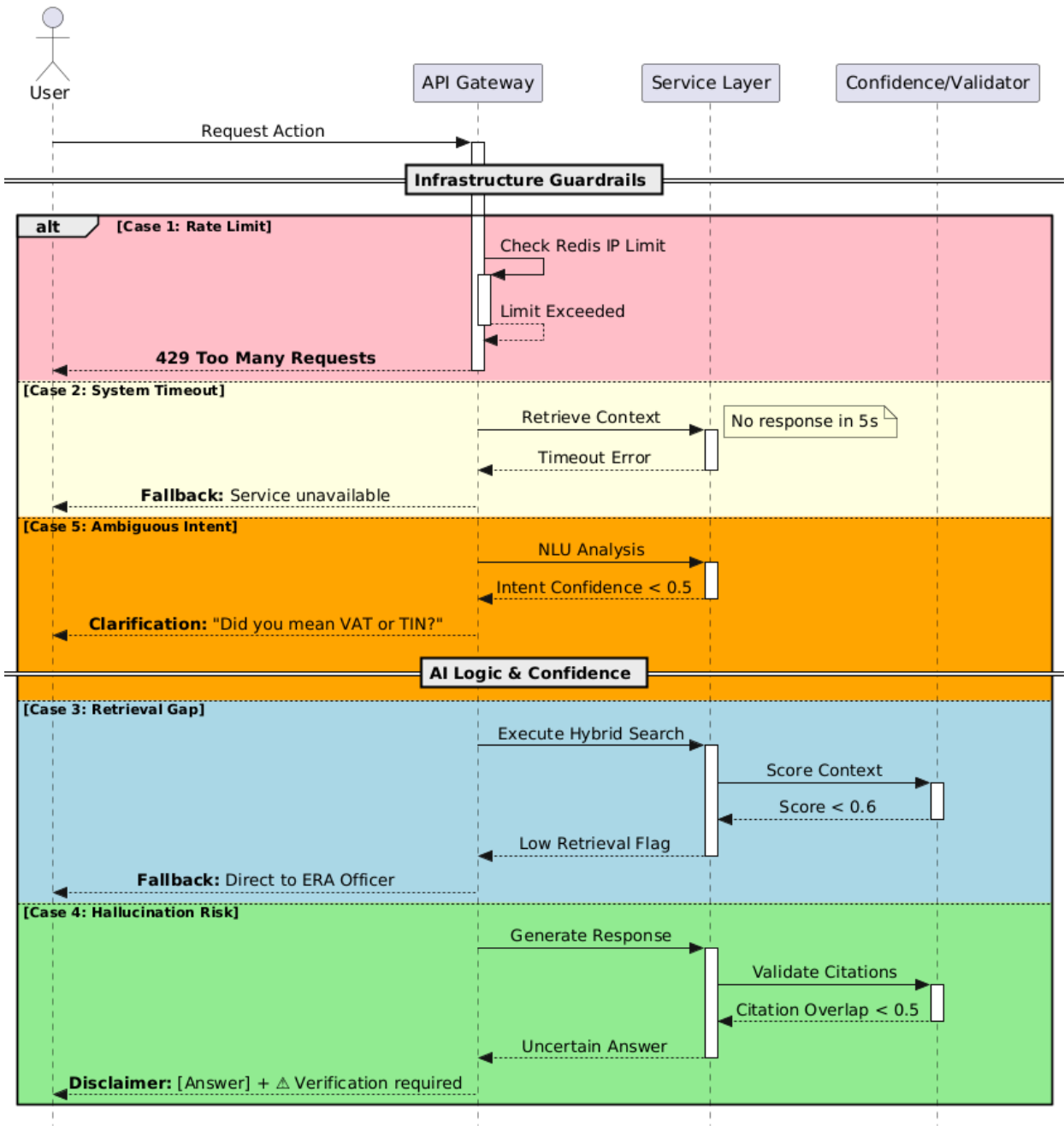


3.2 Sequence Diagram



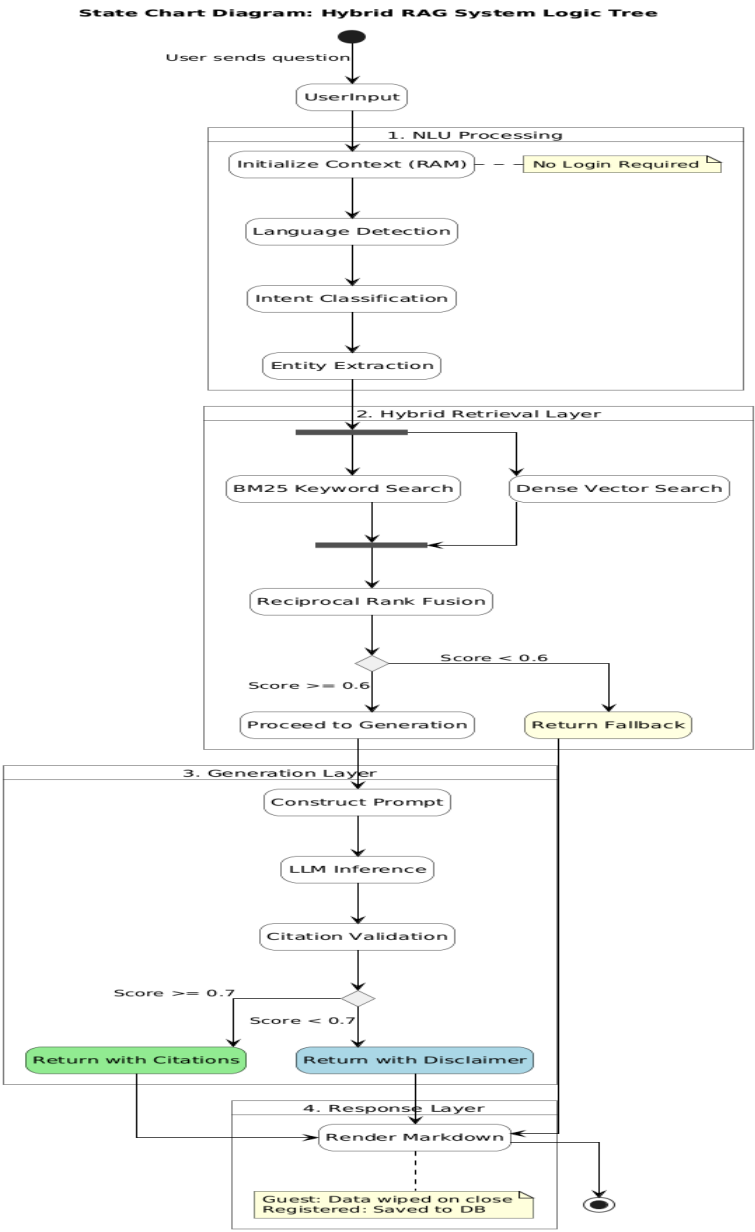
SupportBot AI Next-Gen Revenue Customer Service

Figure 3: Vertical Error & Fallback Strategy



SupportBot AI Next-Gen Revenue Customer Service

3.3 State chartDiagram



SupportBot AI Next-Gen Revenue Customer Service

4. Detailed Design

Table 4.1: Attributes description for Document class

Attribute	Type	Visibility	Invariant
docID	String (UUID)	Private	docID can not be NULL and must be unique across the database.
title	String	Public	title NULL and length > 5 chars. Must match the official legal title.
rawContent	Blob	Private	rawContent NULL. File size must be < 25MB.
fileType	Enum	Public	Must be one of {PDF, DOCX, TXT}.
uploadDate	DateTime	Protected	Defaults to current server timestamp.
processStatus	Enum	Public	Must be {PENDING, PROCESSING, INDEXED, FAILED}.

Table 4.2: Operation description for Document class

SupportBot AI Next-Gen Revenue Customer Service

Operation	Visibility	Return Type	Argument	Pre-Condition	Post-Condition
extractText	Private	String	fileBlob	File must be a valid PDF/DOCX format; File size < 25MB.	Returns raw text string including Amharic Unicode characters.
cleanText	Private	String	rawText	rawText is not null.	Removes headers, footers, and page numbers.
ChunkContent	Public	List<String>	cleanedText	cleanedText length > 100 chars.	Returns list of strings where each string length <= 1024 tokens.
UpdateStatus	Public	Void	newStatus	newStatus is a valid Enum value.	processStatus is updated to the new value.

4.2 Class: Admin (Inherits from User)

SupportBot AI Next-Gen Revenue Customer Service

Represents an authenticated system administrator with privileges to manage the knowledge base manually.

Table 4.3: Attributes description for Admin class

Attribute	Type	Visibility	Invariant
adminID	String	Private	adminID $\diamond$ NULL.
username	String	Public	username must be unique and alphanumeric.
passwordHash	String	Private	Must be SHA-256 encrypted string; never plain text.
role	Enum	Protected	Defaults to SUPER_ADMIN.
sessionToken	String	Private	Valid JWT token with 1-hour expiration.

Table 4.4 Operation description for Admin class

Operation	Visibility	Return Type	Argument	Pre-Condition	Post-Condition
-----------	------------	-------------	----------	---------------	----------------

SupportBot AI Next-Gen Revenue Customer Service

login	Public	Bool	user, pass	the account must exist in PostgreSQL.	If valid, sessionToken is generated; else return False.
Upload Document	Public	Void	fileObj	User must be logged in; File must be valid.	New Document object is created with status PENDING.
viewSystemLogs	Public	JSON	dateRange	User must have a SUPER_ADMIN role.	Returns error/access logs for the specified period.
forceScrape	Public	Void	-	Scraper service must be idle (not currently running).	scraper job is triggered immediately.

4.3 Class ChatSession

Represents a continuous conversation thread between a public user (Taxpayer) and the AI Bot.

SupportBot AI Next-Gen Revenue Customer Service

Table 4.5 Attributes description for ChatSession class

Attribute	Type	Visibility	Invariant
sessionID	String	Public	Unique UUID generated upon first page load.
messageHistory	List<Message>	Private	Ordered list by timestamp; Max size 20 messages (context window).
startTime	DateTime	Private	System time at session creation.
lastActivity	DateTime	Public	lastActivity <= CurrentTime.
clientIP	String	Private	Valid IP address format.

Table 4.6 Operation description for ChatSession class

SupportBot AI Next-Gen Revenue Customer Service

Operation	Visibility	Return Type	Argument	Pre-Condition	Post-Condition
addMessage	Public	Void	msgContent, senderRole	msgContent is not empty; senderRole is {USER, AI}.	Message is appended to messageHistory; lastActivity updated.
getContext	Public	String	-	messageHistory is not empty.	Returns concatenated string of last 5 interactions for LLM context.
checkTimeout	Private	Boolean	-	-	Returns True if CurrentTime - lastActivity > 30 mins.
detectLanguage	Public	String	inputText	inputText is not null.	Returns 'am' (Amharic) or 'en' (English).

4.4 Class KnowledgeEngine (RAG Controller)

SupportBot AI Next-Gen Revenue Customer Service

This is the core logic class responsible for bridging the user query with the Vector Database and the LLM.

Table 4.7 Attributes description for KnowledgeEngine class

Attribute	Type	Visibility	Invariant
embeddingModel	String	Private	Constant: "text-embedding-3-small" or similar.
vectorDBClient	Object	Private	Active connection object to Pinecone.
confidenceThreshold	Float	Public	Value must be between 0.0 and 1.0 (e.g., 0.75).

Table 4.8: Operation description for KnowledgeEngine class

SupportBot AI Next-Gen Revenue Customer Service

Operation	Visibility	Return Type	Argument	Pre-Condition	Post-Condition
vectorizeQuery	Private	Vector[]	queryText	queryText is not empty.	Returns float array representation of text.
retrieveContext	Public	List<String>	queryVector	Vector DB is online.	Returns top 5 document chunks sorted by similarity score.
generateAnswer	Public	String	context, query	LLM API is reachable.	Returns natural language response citing sources.
validateConfidence	Private	Boolean	score		Returns True if score >= confidenceThreshold.

4.5 Class WebScraper

SupportBot AI Next-Gen Revenue Customer Service

The automated agent responsible for keeping the knowledge base up to date.

Table 4.9: Attributes description for WebScraper class

Attribute	Type	Visibility	Invariant
targetURL	String	Private	Constant: " https://mor.gov.et ".
scheduleTime	Time	Private	Constant: 00:00 EAT.
downloadPath	String	Private	Path must be writeable by the system.

Table 4.10: Operation description for WebScraper class

Operation	Visibility	Return Type	Argument	Pre-Condition	Post-Condition
scanForUpdates	Public	List<URL>	-	Internet connection is active.	Returns list of PDF links not present in known_hashes DB.
downloadFile	Protected	File	url	URL is valid.	The file is saved to downloadPath.

SupportBot AI Next-Gen Revenue Customer Service

logError	Private	Void	errorMsg	-	Error details written to system log.
----------	---------	------	----------	---	--

4.6 Algorithm Design

4.6.1 Hybrid Retrieval Augmented Generation (RAG) Flow

The system uses an advanced hybrid RAG pattern that combines keyword matching with semantic search to answer questions based on facts rather than the AI's training memory. This addresses limitations of naive RAG implementations.

1. **Input:** The system receives a query from the user (e.g., "What is the VAT rate?" / "የቫት ምጣኔ ስንት ነው?").
2. **NLU Processing:** Language detection, intent classification (XLM-RoBERTa), and entity extraction (Proclamation numbers, TIN patterns, amounts).
3. **Dual Retrieval Path:**
 - **BM25 Sparse Search:** Keyword matching for exact legal terms, Proclamation numbers (e.g., "285/2002")
 - **Dense Vector Search:** Semantic similarity using multilingual-e5-large embeddings
4. **Fusion & Reranking:** Reciprocal Rank Fusion combines both result sets; Cross-Encoder reranks top candidates.
5. **Confidence Check:** If max retrieval score < 0.6, return fallback message instead of proceeding.
6. **Augmentation:** The system combines the user's question with top-5 retrieved chunks into a structured prompt with citation requirements.
7. **Generation:** The LLM generates a response with mandatory source citations.
8. **Post-Processing:** Citation validation ensures generated claims match source documents.

4.6.2 Vector Similarity Search (Cosine Similarity)

To find the right documents, the system needs to measure how similar a user's question is to the stored documents.

- ### 4.6.3 BM25 Keyword Matching (Sparse Retrieval)

Dense vector search alone may miss exact legal terminology. BM25 provides complementary keyword-based retrieval.

- #### 4.6.4 Reciprocal Rank Fusion (RRF)

Combines BM25 and dense retrieval results into a unified ranking.

Formula: $\text{RRF_score}(d) = \sum 1 / (k + \text{rank_i}(d))$

Where $k=60$ (constant), and $\text{rank}_i(d)$ is the rank of document d in retrieval system i .

SupportBot AI Next-Gen Revenue Customer Service

Benefit: Documents appearing in top ranks of both systems receive the highest combined scores, leveraging strengths of both approaches.

4.6.5 Text Chunking Algorithm (Sliding Window)

Tax documents are often very long, but the AI can only read a certain amount of text at once. To solve this, the system breaks documents into smaller pieces.

- 1. **Chunking:** The document is split into blocks of approximately 750 words (1024 tokens).
- 2. **Overlap:** To prevent cutting a sentence in half at the end of a block, the system creates an "overlap." The last 100 words of Block A are repeated as the first 100 words of Block B. This ensures the context flows smoothly from one chunk to the next.
- 3. **Metadata Preservation:** Each chunk retains source document ID, page number, and section header for citation purposes.
- 4. **4.6.6 Confidence Estimation Algorithm**

The system implements multi-signal confidence scoring to prevent hallucinations.

Signal	Weight	Measurement
Retrieval Score	0.4	Max cosine similarity from dense search
BM25 Score	0.2	Normalized BM25 score
Citation Overlap	0.3	Ratio of generated claims with source support
LLM Uncertainty	0.1	Token probability variance (if available)

SupportBot AI Next-Gen Revenue Customer Service

Decision Logic:

- Combined Score ≥ 0.7 → Return answer with citations
- Combined Score 0.5-0.7 → Return answer with disclaimer
- Combined Score < 0.5 → Return fallback: "Please consult an ERA officer"

4.6.7 Guest Rate Limiting Algorithm (Token Bucket)

To secure the free tier, the system implements a volatile rate limiter using Redis:

- **Identifier:** Client_IP_Address
- **Storage:** Redis Cache (In-Memory)
- **Logic:**
 - Check: `GET rate_limit:{IP}`
 - If Null: `SET rate_limit:{IP} = 1, EXPIRE 600` (10 minutes)
 - If < 15 : `INCR rate_limit:{IP}`. Allow request.
 - If ≥ 15 : Return HTTP 429 (Too Many Requests)
- **Privacy:** The Redis key automatically expires after 10 minutes, ensuring no permanent log of the IP address is retained.

4.6.8 Intent Classification Algorithm

Using XLM-RoBERTa in zero-shot classification mode:

Intent Categories:

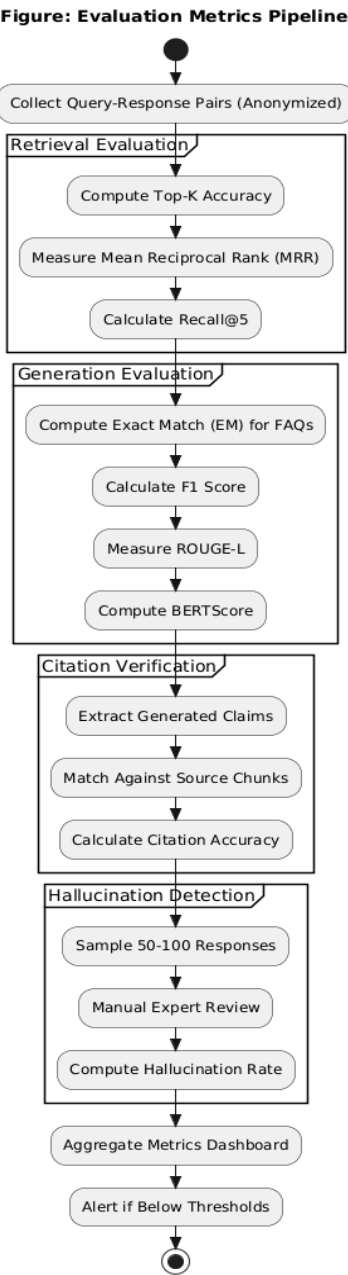
1. **Informational** - "What is VAT?" → Retrieve definition documents
2. **Procedural** - "How do I register for TIN?" → Retrieve step-by-step guides
3. **Definitional** - "Define withholding tax" → Retrieve glossary entries
4. **Clarification** - "Can you explain step 3?" → Use session context

Prompt Template for Zero-Shot:

1. Classify the following query into one of: Informational, Procedural, Definitional, Clarification
2. Query: {user_query}
3. Classification

4.7 Evaluation Framework Design

4.7.1 Automated Metrics Pipeline



SupportBot AI Next-Gen Revenue Customer Service

4.7.2 Evaluation Metrics Definitions

Metric	Formula/Description	Target
Top-K Retrieval Accuracy	% of queries where correct doc is in top-K	$\geq 85\%$ (K=5)
Exact Match (EM)	% of answers exactly matching ground truth	$\geq 70\%$ (FAQs)
F1 Score	Harmonic mean of precision and recall	≥ 0.8
ROUGE-L	Longest common subsequence overlap	≥ 0.5
BERTScore	Semantic similarity of generated vs reference	≥ 0.85
Citation Accuracy	% of citations correctly pointing to source	$\geq 95\%$
Hallucination Rate	% of responses with unsupported claims	$< 5\%$
Multilingual Parity	Accuracy difference (Amharic vs English)	$\leq 5\%$

SupportBot AI Next-Gen Revenue Customer Service

4.8 Key Algorithms & AI/ML Architecture

This section defines the core algorithmic components that power the system's intelligence.

4.8.1 Natural Language Understanding (NLU) Pipeline

Purpose: Understand user queries in Amharic, English, or mixed code-switched input.

Component	Algorithm/Model	Justification
Language Detection	fastText langdetect	Lightweight, supports Amharic, no training required
Tokenization	SentencePiece (Amharic), SpaCy (English)	Handles Ethiopic script morphology
Intent Classification	XLM-RoBERTa (zero-shot)	Pre-trained multilingual model with Amharic support
Entity Extraction	Gemini 2.5 Flash (structured prompts)	More robust than RegEx for tax-specific entities

SupportBot AI Next-Gen Revenue Customer Service

4.8.2 Information Retrieval (Hybrid RAG)

Purpose: Retrieve relevant ERA documents to ground responses and prevent hallucinations.

Component	Algorithm	Description
Sparse Retrieval	PostgreSQL Full-text Search	Keyword matching for exact legal terms, Proclamation numbers
Dense Retrieval	Cosine Similarity on Embeddings	Semantic search using multilingual-e5-large embeddings
Fusion	Reciprocal Rank Fusion (RRF)	Combines sparse and dense retrieval scores
Reranking	Cross-Encoder (ms-marco-MiniLM)	Final relevance scoring of top candidates

4.8.3 LLM Response Generation

Purpose: Generate accurate, readable, and properly cited answers.

The system uses Gemini 2.5 Flash as the primary language model for generating responses. All responses must include proper source citations with specific Proclamation numbers and article references to ensure legal accuracy and traceability.

SupportBot AI Next-Gen Revenue Customer Service

4.8.4 Confidence Estimation & Fallback

Purpose: Determine response reliability and trigger human escalation when needed.

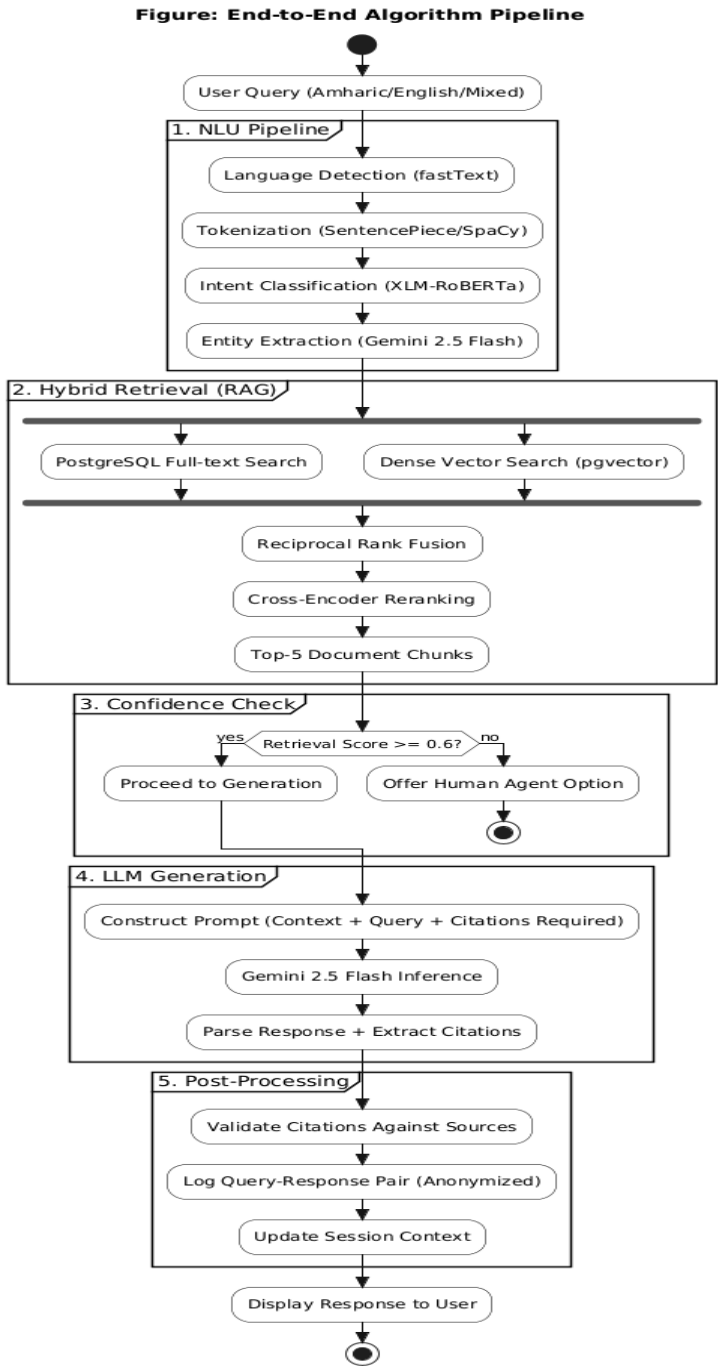
The system uses retrieval confidence scoring based on the maximum cosine similarity score from the document search. When the retrieval confidence falls below 0.6, the system will inform the user that it cannot find sufficient information and offer the option to contact a human ERA officer for assistance.

4.8.5 Dialogue Management

Purpose: Maintain conversation flow and multi-turn context.

1. Context Window: Last 5 exchanges (user + assistant) maintained in session
2. Slot Filling: Track taxpayer category, query topic, and pending clarifications
3. Handoff Management: When conversation is escalated to human agents, provide a summarization of the chat history and tracked user slots

4.8.6 Algorithm Flow Diagram



4.9 Model Selection Rationale

4.9.1 LLM Selection

Decision: Gemini 2.5 Flash

Criterion	Gemini 2.5 Flash	Justification
Amharic Support	Excellent	Strong multilingual capabilities
Cost	Moderate	Cost-effective for production scale
Latency	~1-2s	Fast inference suitable for real-time chat
API Reliability	High	Google Cloud infrastructure
Recommendation	Primary Choice	Best balance of performance, cost, and reliability

SupportBot AI Next-Gen Revenue Customer Service

4.9.2 Database Stack Selection

Decision: PostgreSQL + pgvector

Component	Technology	Justification
Vector Storage	pgvector extension	Native PostgreSQL integration, no separate vector DB needed
Full-text Search	Elasticsearch	simpler stack
Relational Data	PostgreSQL	Unified database for all data types
Advantage	Simplified Stack	Single database technology reduces complexity and maintenance

SupportBot AI Next-Gen Revenue Customer Service

4.9.3 Fine-Tuning vs Prompt Engineering

Decision: Prompt Engineering + RAG (No Fine-Tuning)

Factor	Fine-Tuning	Prompt Engineering	Our Choice
Amharic Data Availability	Insufficient	Not required	Prompt Engineering
Update Frequency	Requires retraining	Update prompts only	Prompt Engineering
Computational Cost	High	Low	Prompt Engineering
Iteration Speed	Slow	Fast	Prompt Engineering
Factual Grounding	Model memory	RAG retrieval	RAG

5. References

SupportBot AI Next-Gen Revenue Customer Service

1. IEEE Standards

- *IEEE Std 1016-2009: IEEE Recommended Practice for Software Design Descriptions.*

2. Academic & Conceptual Foundations

- Lewis, P., et al. (2020). *Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks*. arXiv preprint arXiv:2005.11401. (Foundational paper for the RAG architecture used in Section 4.6.1).
- Robertson, S., & Zaragoza, H. (2009). *The Probabilistic Relevance Framework: BM25 and Beyond*. Foundations and Trends in Information Retrieval.
- Conneau, A., et al. (2020). *Unsupervised Cross-lingual Representation Learning at Scale (XLM-RoBERTa)*. arXiv:1911.02116.

3. Technical Documentation (Backend & Infrastructure)

- FastAPI: *FastAPI Web Framework Documentation*. Retrieved from <https://fastapi.tiangolo.com/>
- Docker: *Docker Engine User Guide & Container Best Practices*. Retrieved from <https://docs.docker.com/>
- PostgreSQL: *pgvector Extension Documentation*. Retrieved from <https://github.com/pgvector/pgvector>
- LangChain: *Building RAG Pipelines with LLMs*. Retrieved from <https://python.langchain.com/>
- PostgreSQL: *Full-Text Search Documentation*. Retrieved from <https://www.postgresql.org/docs/current/textsearch.html>

4. Technical Documentation (Frontend)

- Next.js: *Next.js Documentation*. Retrieved from <https://nextjs.org/docs>
- React: *React.js Documentation*. Retrieved from <https://react.dev/>
- TypeScript: *TypeScript Language Specification*. Retrieved from <https://www.typescriptlang.org/docs/>

5. AI/ML Resources

- Google AI: *Gemini API Documentation*. Retrieved from <https://ai.google.dev/gemini-api>
- Hugging Face: *XLM-RoBERTa Model Card*. Retrieved from <https://huggingface.co/xlm-roberta-base>
- *multilingual-e5-large Model Card*. Retrieved from <https://huggingface.co/intfloat/multilingual-e5-large>
- Telegram: *Bot API Documentation*. Retrieved from <https://core.telegram.org/bots/api>

SupportBot AI Next-Gen Revenue Customer Service

SupportBot AI Next-Gen Revenue Customer Service
